

Name : YASHVI MEHTA

Roll no. : 231071079

LY BTech Computer

Batch D

Cross Platform App Dev Lab 5

AIM -

To study the concept of the Document Object Model (DOM) and develop a simple HTML DOM application using the Dart programming language, understanding how Dart can dynamically access, manipulate, and update the structure, content, and style of a web page as part of cross-platform application development.

THEORY -

Document Object Model (DOM) -

The **Document Object Model (DOM)** is a programming interface used to represent HTML and XML documents in a structured form. When a web browser loads an HTML page, it parses the HTML elements and creates a tree-like structure known as the **DOM tree**. Each HTML element, attribute, and text content is represented as a **node** in this tree.

The DOM allows programs to access and manipulate the **structure, content, and styling** of a web page dynamically. For example, a program can change the text of an element, modify its style, add or remove elements, or respond to user interactions such as button clicks.

Since the DOM exists in the browser's memory, modifications made to it are immediately reflected on the webpage **without requiring a complete page reload**. This makes DOM manipulation an important foundation for creating interactive web applications.

Dart and dart:html -

Dart is an object-oriented, general-purpose programming language developed by Google. It can be used for developing applications for multiple platforms, including web, mobile, desktop, and server environments.

For web applications, Dart code can be **compiled into JavaScript** so that it can execute inside modern web browsers. Dart provides the **dart:html** library for interacting with browser features and the HTML DOM.

The **dart:html** library provides Dart representations of common browser objects such as the document, HTML elements, and events. It allows developers to select HTML elements, read or modify their properties, register event listeners, and dynamically change the contents of a webpage.

The dart:html library provides Dart representations of common browser objects such as the document, HTML elements, and events. It allows developers to select HTML elements, read or modify their properties, register event listeners, and dynamically change the contents of a webpage.

For example, an element can be selected using methods such as **querySelector()**, and its text or style can then be modified using Dart code.

DOM Manipulation using Dart -

DOM manipulation refers to programmatically accessing and changing elements of an HTML document. In a Dart web application, the **document object** represents the currently loaded HTML page and acts as the starting point for accessing DOM elements.

The major operations involved in DOM manipulation are:

- **Selecting elements:** HTML elements can be located using their IDs, classes, tags, or other CSS selectors using methods such as `querySelector()` and `querySelectorAll()`.

- **Reading and modifying content:** The text and innerHtml properties can be used to read or change the content of an element.
- **Modifying styles:** The style property allows properties such as colour, background colour, font size, and visibility to be changed dynamically.
- **Managing CSS classes:** The classes property can be used to add, remove, or toggle CSS classes associated with an element.
- **Creating and removing elements:** New elements can be created and added to the DOM, while existing elements can be removed when required.

Event Handling -

Event handling is an important part of interactive web applications. An **event** is an action that occurs in the browser, such as a mouse click, keyboard input, form submission, or change in an input field.

Dart provides event-listening mechanisms through the **dart:html** library. For example, a button's click event can be monitored using an event listener. When the user clicks the button, the associated Dart function is executed.

The event-handling process can therefore be summarized as:

User Action → Event Generated → Event Listener Triggered → Dart Function Executed → DOM Updated

This allows applications to provide immediate feedback to users without reloading the webpage.

Important dart:html Classes and Methods -

Class / Method	Purpose
document	Represents the current HTML document and provides access to its DOM.

Class / Method	Purpose
<code>querySelector()</code>	Selects the first element matching a CSS selector.
<code>querySelectorAll()</code>	Selects all elements matching a CSS selector.
<code>Element</code>	Represents an HTML element selected from the DOM.
<code>text</code>	Reads or modifies the text content of an element.
<code>innerHTML</code>	Reads or modifies the HTML content inside an element.
<code>style</code>	Provides access to an element's inline CSS properties.
<code>classes</code>	Allows CSS classes to be added, removed, or toggled.
<code>onClick</code>	Provides an event stream for mouse-click events.
<code>on[event].listen()</code>	Registers a listener for a specified browser event.
<code>append()</code>	Adds a new node or element to the DOM.
<code>remove()</code>	Removes an element from the DOM.

Role of HTML and Dart in the Application -

In a Dart HTML DOM application, **HTML provides the basic structure** and user interface of the webpage, while **Dart provides the application logic** and interaction.

In a Dart HTML DOM application, HTML provides the basic structure and user interface of the webpage, while Dart provides the application logic and interaction.

The index.html file contains elements such as headings, paragraphs, input fields, and buttons. These elements are assigned unique IDs or classes so that they can be identified from Dart code.

The Dart entry-point file contains the logic for selecting these elements and responding to user actions. When an event occurs, the corresponding Dart function **modifies the required DOM element**. The browser then immediately displays the updated content.

The Dart entry-point file contains the logic for selecting these elements and responding to user actions. When an event occurs, the corresponding Dart function modifies the required DOM element. The browser then immediately displays the updated content.

Thus, the basic workflow is:

HTML Structure → Dart Element Selection → Event Handling → DOM Manipulation → Updated Webpage

HTML Structure → Dart Element Selection → Event Handling → DOM Manipulation → Updated Webpage

Cross-Platform Relevance of Dart -

Dart supports application development across multiple platforms. For web applications, Dart can target the browser and interact with HTML through browser APIs. Dart is also commonly used with the Flutter framework for developing applications for Android, iOS, desktop, and other supported platforms.

APP OVERVIEW -

Text Content Change - You can type any text in the input box and click "Update Text". Dart finds the paragraph on the screen and changes its text immediately. You can also click "Reset" to return to the original text.

Style & Color Change - You can click different color buttons (Blue, Green, Purple, Rose, Amber) to change the background and borders of a box, or change font size and bold formatting using Dart.

Theme Toggling (CSS Classes) - You can click buttons like "Success", "Warning", "Danger", or "Dark" to dynamically switch visual themes by having Dart apply and remove CSS classes.

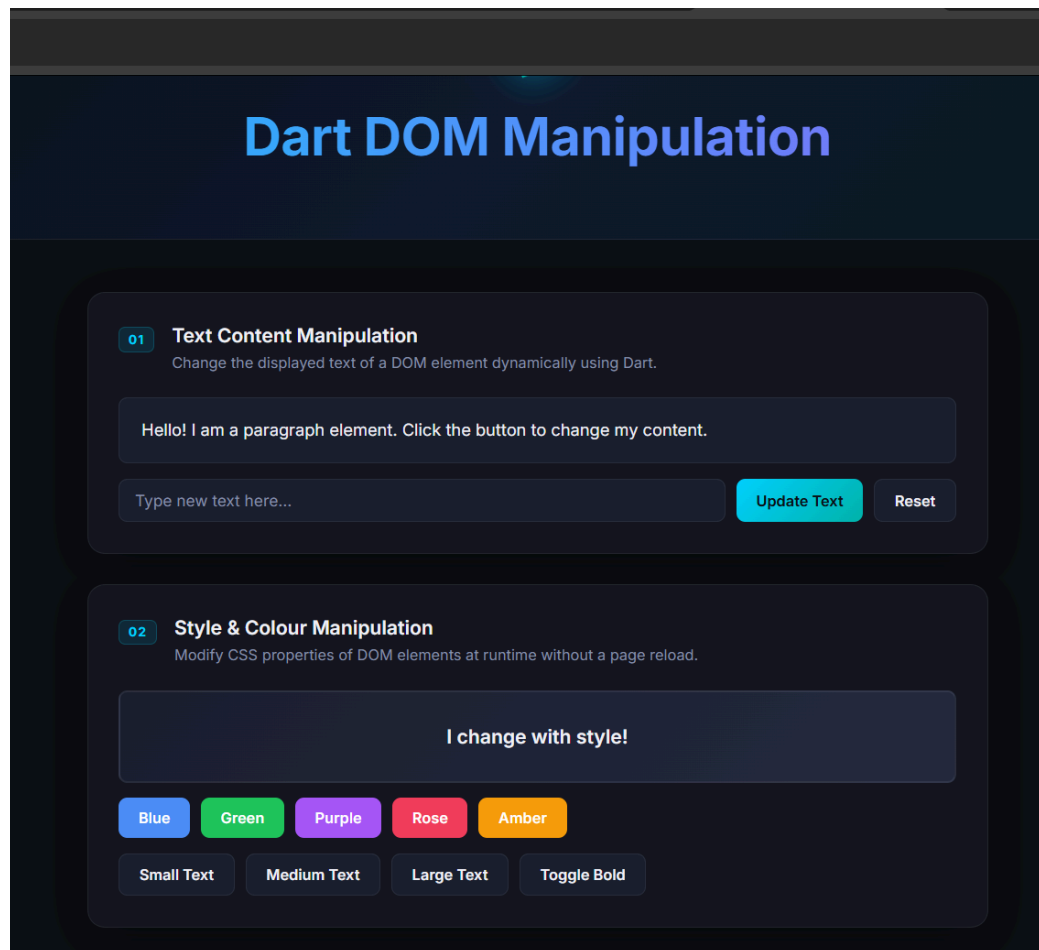
Click Counter & Event Log - You can increment, decrement, or reset a counter. Every time you click a button, Dart updates the counter number with an animation and writes a timestamped record in the live event log below.

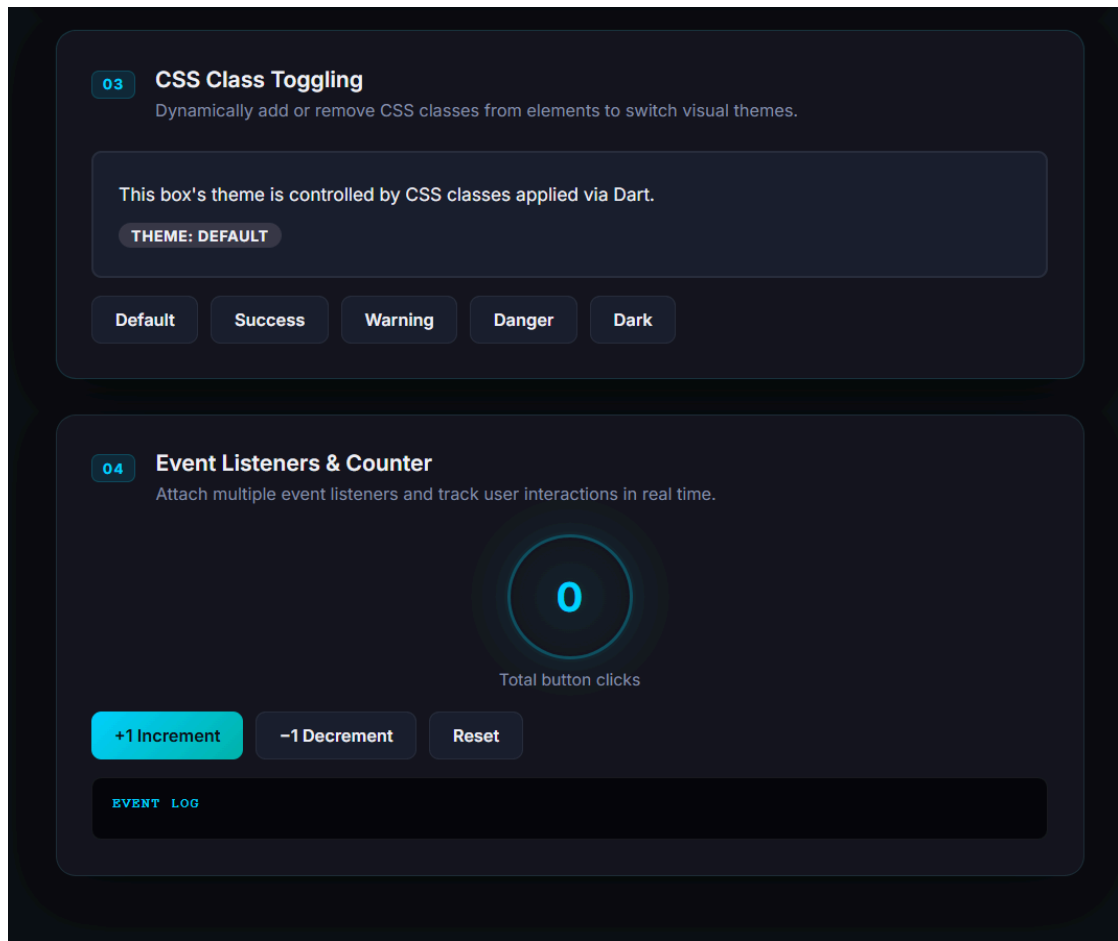
REPO LINK FOR CODE -

https://github.com/YashviMehta03/CPAD_lab_repos/tree/main/lab5_app

SCREENSHOTS -

App -

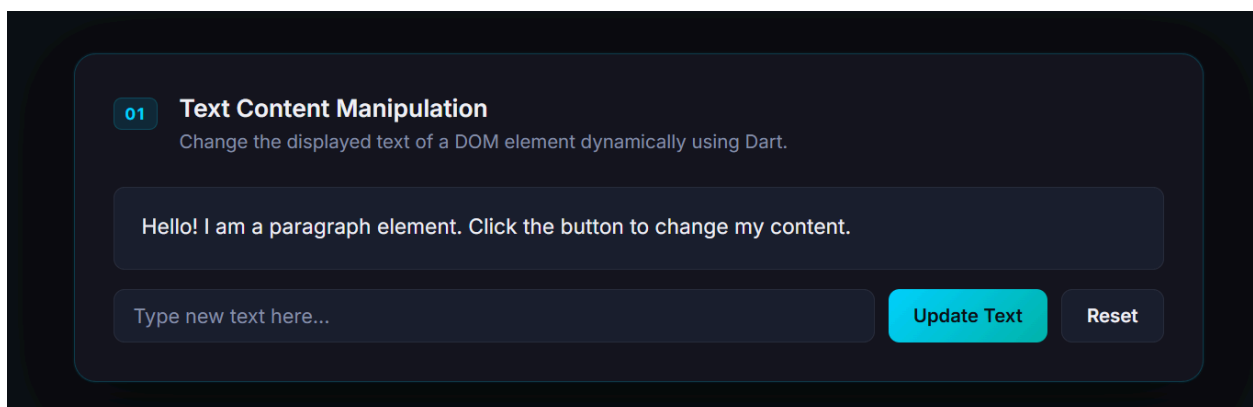




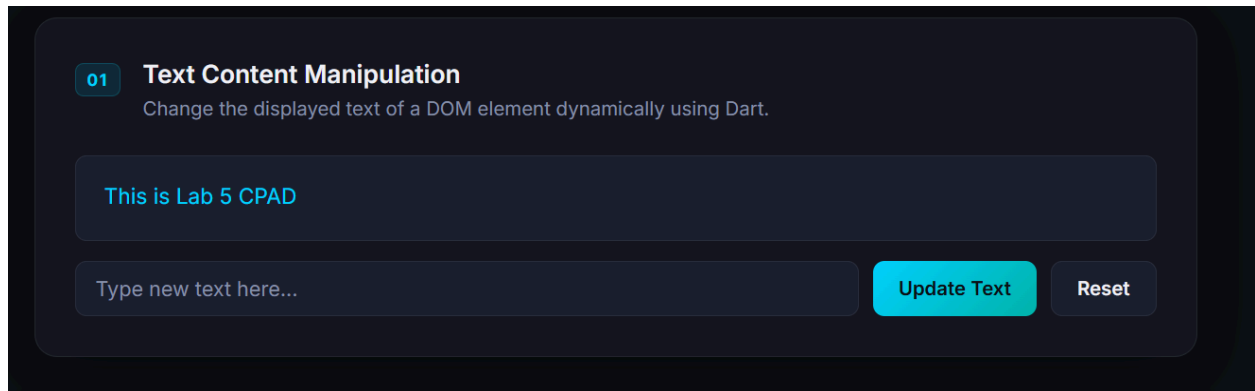
Each functionality -

1) Text Content Manipulation

Initial ->

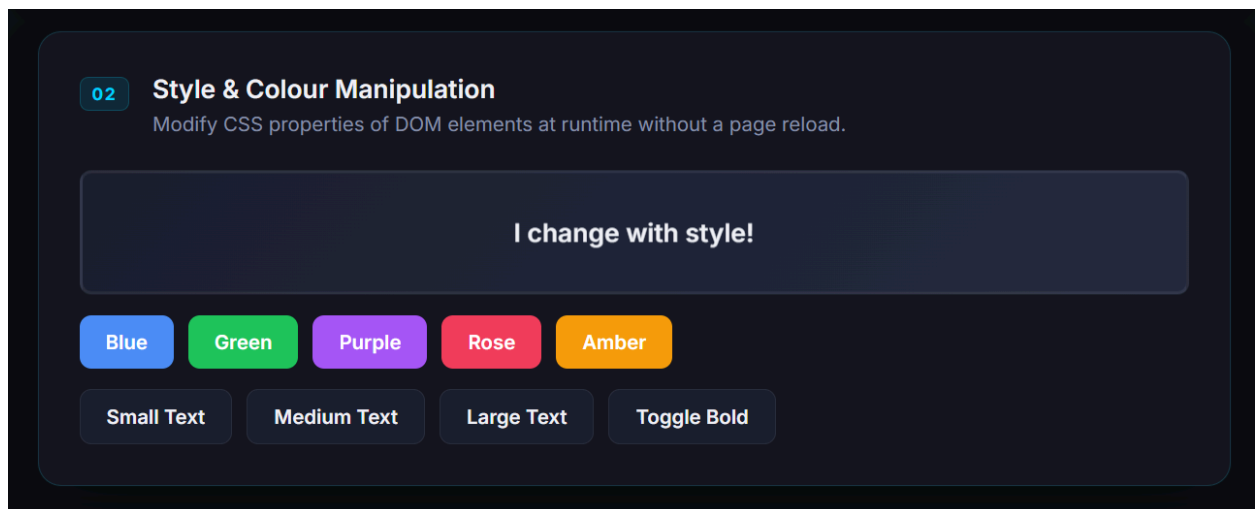


On updation ->

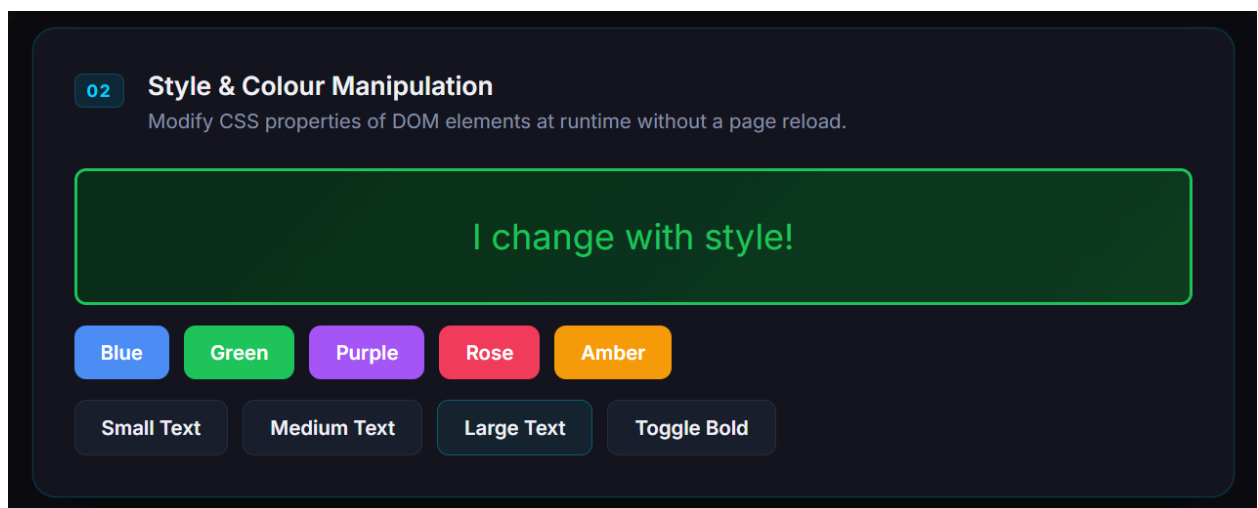


2) Style & Colour Manipulation

Initial ->

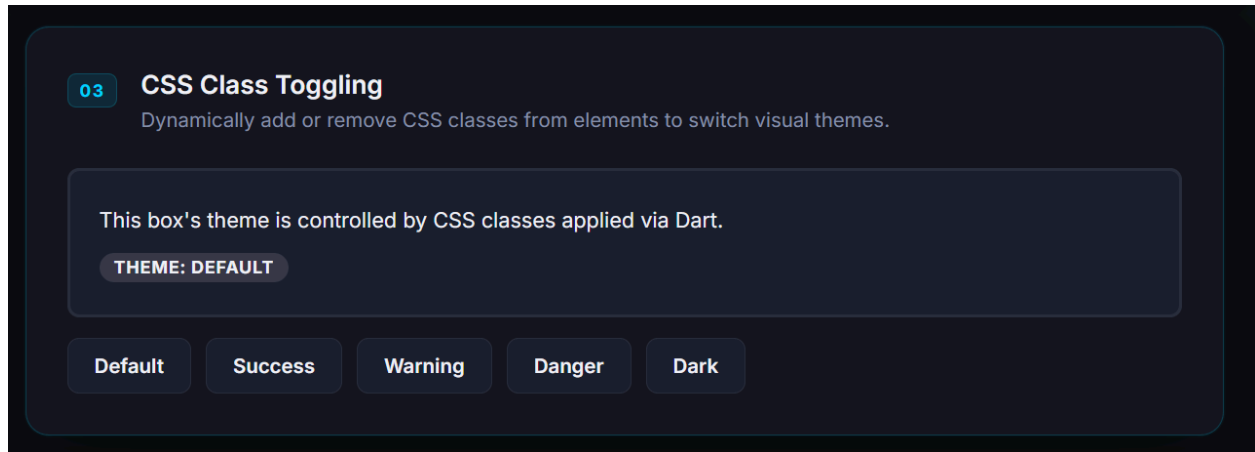


On updation ->

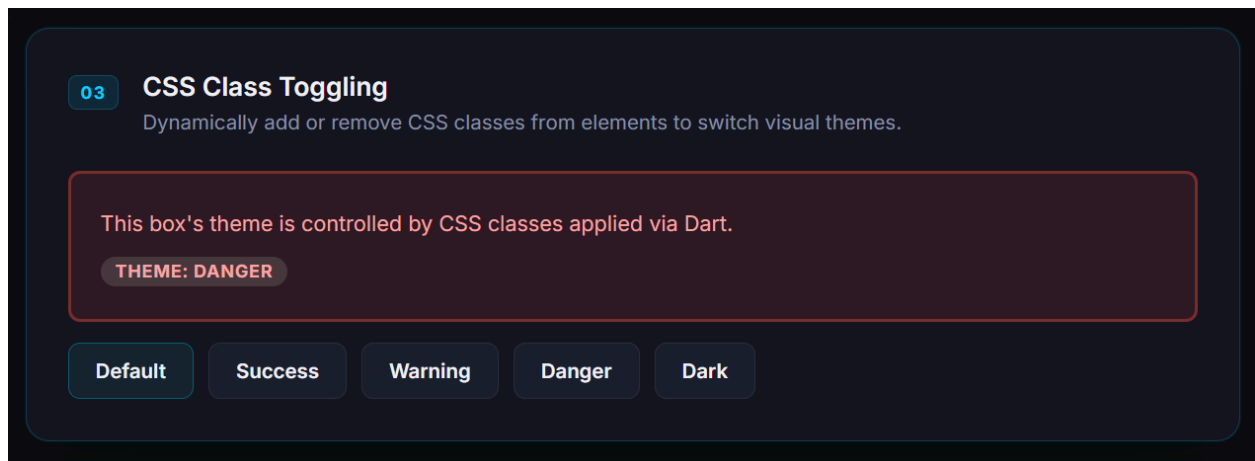


3) CSS Class Toggling

Initial ->

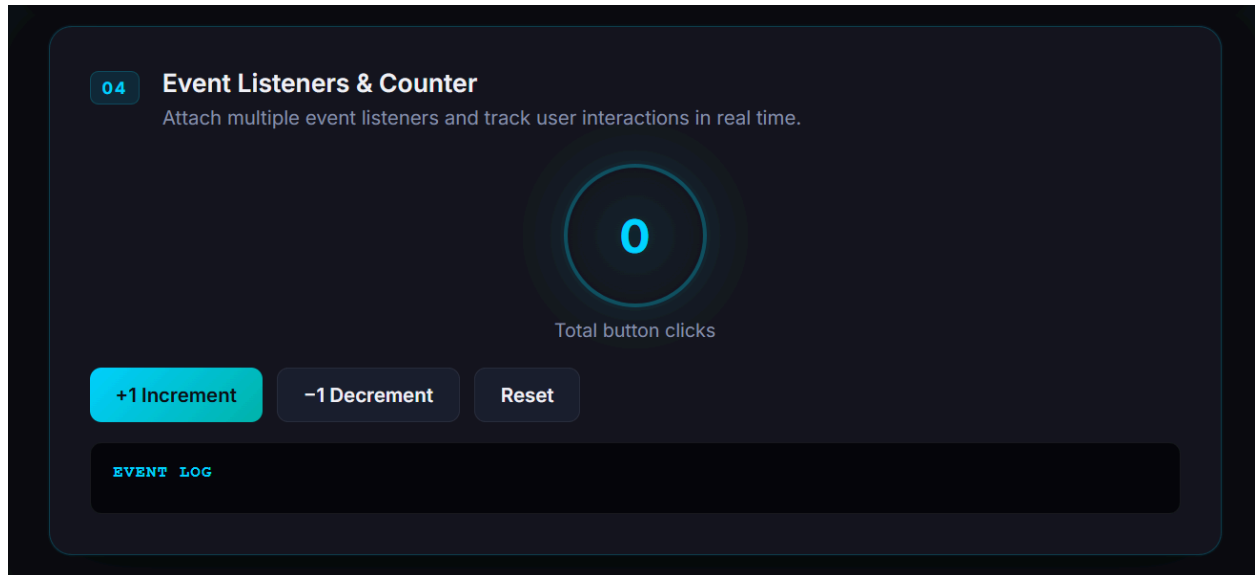


On updation ->

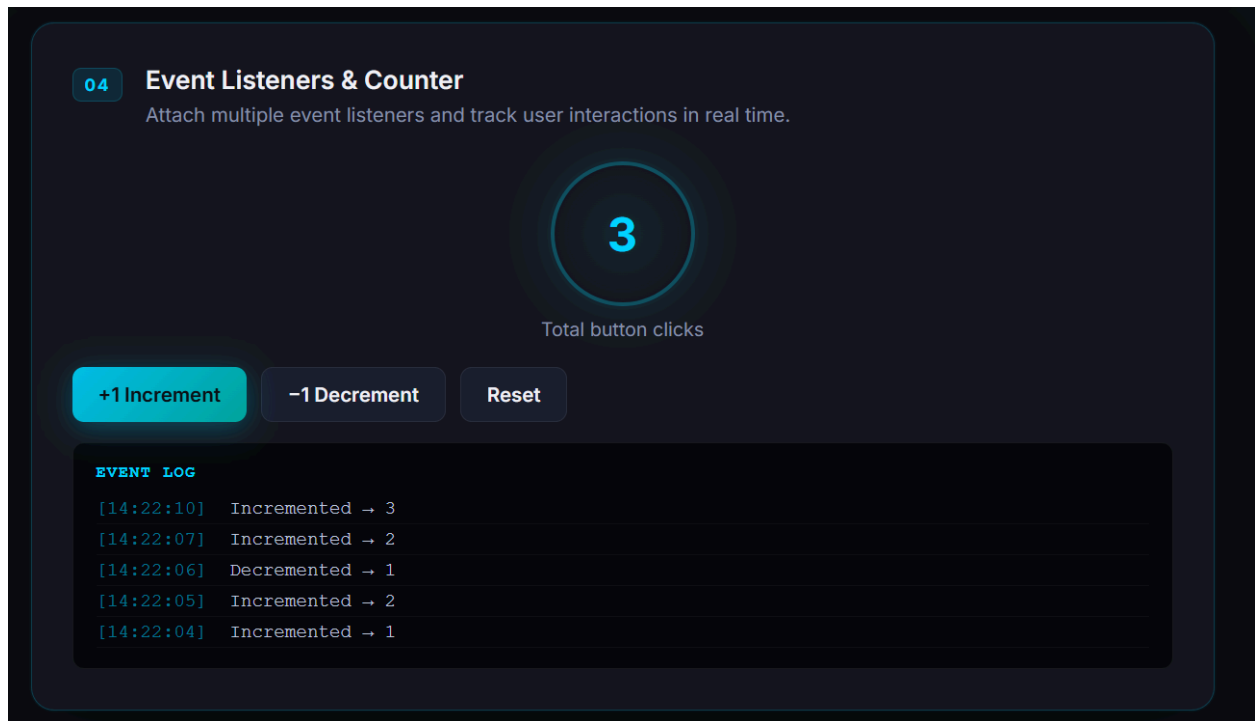


4) Event Listeners & Counter

Initial ->



On updation ->



CONCLUSION -

The concept of the Document Object Model (DOM) and its role in developing interactive web applications was studied and implemented using Dart. A simple HTML DOM application was developed in which Dart accessed HTML elements, handled user-generated events, and dynamically modified the webpage content and style. The experiment demonstrated how the `dart:html` library enables communication between Dart code and the browser DOM, allowing web pages to be updated without reloading.